

- Passwords are hashed with bcryptjs using 10 rounds; password hashes are excluded from API responses.
- JWTs carry and validate authenticated identity and role information; client-supplied user IDs cannot replace the authenticated identity.
- Role checks are enforced server-side rather than relying on hidden or disabled frontend controls.
- Atomic database updates protect inventory from negative stock and concurrent over-reservation.
- Idempotency protections prevent duplicate processing of sensitive inventory mutations.
- Centralized error handling returns sanitized structured errors.
- Environment-based configuration keeps deployment-specific settings outside business logic.
- Database schema and migration status were verified during the final release review.

The case study explicitly requires environment-based configuration and says the business logic should remain portable rather than being tightly coupled to a hosting provider. [filecite\turn3file2\L239-L240](#)

17. Setup and Operations

The repository is organized into backend and frontend applications. The project walkthroughs use the following development and verification commands.

Purpose	Command
Backend tests	cd backend && npx vitest run
Backend type check	cd backend && npx tsc --noEmit
Backend build	cd backend && npm run build
Frontend type check	cd frontend && npx tsc --noEmit
Frontend build	cd frontend && npm run build
Prisma validation	cd backend && npx prisma validate
Migration status	cd backend && npx prisma migrate status

The case study's README submission requirement calls for the technology stack, project setup, database setup, environment variables, how to run and how to test. [filecite\turn3file2\L262-L274](#)

18. Release Readiness

The Phase 9 verification identifies the project as release-ready. The reported evidence includes complete Swagger/OpenAPI documentation, a 19-endpoint API inventory, RBAC/security review, database validation and migration status, 115/115 tests passing, successful TypeScript checks and successful production builds.

Check	Reported result
Backend tests	115 / 115 passed
Backend TypeScript	0 errors
Backend production build	Passed
Frontend TypeScript	0 errors
Frontend production build	Passed
Database schema validation	Passed
Migration status	Up to date; 1 migration applied
API documentation	OpenAPI 3.0 / Swagger UI
Git state	Working tree clean

Check	Reported result
Release state	Approved & release ready (Phase 9 report)

19. Phase Completion Summary

Phase	Scope	Verified outcome
Phase 2	Authentication & RBAC	17/17 tests passed
Phase 3	Inventory engine	17/17 tests passed
Phase 4	Work Orders + shortage	19/19 tests passed
Phase 5	Internal transfers	23/23 tests passed
Phase 6	Customer orders + reservation	22/22 tests passed
Phase 7	Production frontend	TypeScript/build passed; integrated UI delivered
Phase 8	Comprehensive edge testing	115/115 total active tests passed
Phase 9	API docs + hardening + release	Swagger, security, DB and release checks passed

The repository history was developed incrementally rather than submitted as a single final commit, consistent with the case study's Git requirement for reasonable development history. [filecite turn3file2 L281-L286](#)

20. Assumptions and Limitations

- This document records the implementation and verification evidence supplied in the project walkthroughs; it does not independently re-run the application.
- Local development URLs such as localhost:5000 and localhost:3000 are environment-specific and are not production deployment addresses.
- The case study lists several possible live-verification changes, including damaged stock, partial receipt, cancellation/release and location restrictions. These are examples of future evaluation changes, not all required baseline features. [filecite turn3file2 L296-L310](#)
- A production deployment should still supply environment-specific secrets, database credentials, HTTPS, hosting configuration, monitoring and operational backup procedures appropriate to the chosen infrastructure.
- The case study requires a short 5–7 minute demonstration video showing Login → Inventory → Work Order → Transfer → Order Reservation; this document does not substitute for that video deliverable. [filecite turn3file2 L274-L280](#)

21. Conclusion

The Mini Operations ERP implements the core operations workflow defined by the Technical Case Study: authenticated role-based access, multi-location inventory, Work Orders with automatic shortage calculation, internal stock movement and customer stock reservation.

The strongest technical characteristics are the server-side RBAC model, relational data integrity, transaction boundaries around stock mutations, atomic PostgreSQL conditional updates for concurrency-sensitive operations, explicit lifecycle validation, idempotency protection and a growing automated test suite.

The final project verification supplied for this documentation reports 115/115 active backend tests passing, clean TypeScript and production builds for both applications, verified database migrations, documented APIs and a completed security/release-readiness review. On that evidence, the project satisfies the stated technical scope and is positioned for final submission and demonstration.

Reference

Full-Stack Developer Technical Case Study — Mini Operations ERP, supplied with the project. The case study defines the required business scenario, mandatory modules, technical requirements, minimum screens, tests, submission requirements and evaluation criteria. ■filecite■turn3file0■L12-L30■